# ASSIGNMENT – 2

# Software Testing and Quality Assurance

MCA – 2nd Year

Case Studies and Practical Test Design

Name: _____

Roll No.: _____

Department: _____

## 1. Develop a complete testing strategy for an Online Examination Management System.

Introduction: The system allows students to register, select subjects, pay fees, download admit cards, and view results. The strategy should confirm that these functions meet requirements, protect student information, and work reliably during peak examination periods.

Testing fundamentals:

- Testing shows the presence of defects; it cannot prove that no defects exist.
- Testing should start early and be planned according to risk.
- Exhaustive testing is impractical, so critical workflows and boundary cases receive priority.
- Defects often cluster in complex or frequently changed areas.
- Testing must be adapted to the system and its users.

Testing at different stages:

1. Requirement stage — Static testing and verification: Review requirements for ambiguity, missing rules, payment/refund conditions, subject eligibility, and result visibility. This prevents misunderstandings before coding.
2. Design stage — Reviews and risk analysis: Review database design, architecture, security, payment integration, and role permissions. This detects design flaws early.
3. Development stage — Static analysis and unit testing: Review code and run static-analysis tools. Developers unit-test registration rules, fee calculations, subject selection, and result calculations.
4. Integration stage — Integration testing: Check connections between registration, payment gateway, admit-card generation, and result modules. Verify correct data exchange and payment status handling.
5. System stage — Dynamic testing: Test the complete workflow, functional behaviour, usability, performance, security, compatibility, and recovery from failures.
6. Acceptance stage — Validation/UAT: Students, examination staff, and administrators confirm the system supports real examination tasks.
7. Release stage — Regression and smoke testing: Recheck critical flows after changes and verify the deployed build.

Verification asks, “Are we building the product right?” Validation asks, “Are we building the right product?” Both are required: verification checks conformance to specifications, while validation checks actual user needs.

Important test types:

- Functional: registration, subject selection, payments, admit cards, and results.
- Security: unauthorized access, data privacy, session management, and role permissions.
- Performance/load: many students registering or viewing results at once.
- Usability/accessibility: clear forms and accessible navigation.
- Compatibility: supported browsers, devices, and operating systems.
- Recovery/reliability: network loss, failed payments, duplicate submissions, and server restart.

Entry/exit criteria and reporting: Begin a test phase when requirements/builds/environments are ready. Complete it when planned high-risk tests are executed, critical defects are resolved or formally accepted, and remaining risks are reported. This layered strategy finds defects early, reduces release risk, and provides evidence that the system is fit for use.

## 2. Identify defects in an online banking application detectable through static testing and explain cost savings.

Static testing examines requirements, designs, code, and other work products without executing the application.

Possible defects:

1. Requirements: unclear transfer limits, missing rules for failed transfers, ambiguous beneficiary verification, or unspecified transaction-history retention.
2. Design: no atomic transaction handling, inadequate audit logging, weak session design, or no plan for duplicate requests.
3. Security: passwords or account data logged in plain text, hard-coded credentials, missing authorization checks, weak cryptographic choices, or secrets stored in source code.
4. Code: incorrect balance calculations, duplicated transfer logic, unreachable code, unsafe null handling, race conditions, and incorrect error handling.
5. Database: missing constraints, incorrect data types, missing indexes, or transactions that could update one account without the other.
6. Interface/API: inconsistent response codes, missing input validation, or exposed sensitive information in error messages.
7. Maintainability: misleading names, complex functions, duplicated code, and missing explanations for non-obvious business rules.

Static methods include requirement reviews, design inspections, peer code reviews, coding-standard checks, and static-analysis/security tools.

Cost reduction: A defect found in a requirement or design can be corrected before code, integration, and test data depend on it. Static analysis can identify risky patterns quickly. This reduces repeated debugging, retesting, rework, and late-stage failures. Static testing complements dynamic testing; it cannot reveal every runtime problem, such as actual response-time issues or environment-specific failures.

## 3. Design a formal review process for a 2,000-line module.

Objective: Review the module systematically for correctness, consistency, maintainability, security, and compliance with coding standards.

Participants:

- Moderator/review leader: plans the review and keeps it focused.
- Author: explains the module and corrects defects.
- Reviewers: developers with relevant technical knowledge.
- Recorder/scribe: records defects and decisions; this role may be combined if appropriate.
- Optional test/security/domain specialist for high-risk logic.

Preparation:

1. Confirm the module scope, purpose, requirements, and coding standards.
2. Divide the 2,000 lines into manageable sections and review in multiple sessions.
3. Share the code, design notes, test cases, and checklist beforehand.
4. Reviewers inspect independently and record suspected defects.
5. Set a time limit and define defect categories and severity levels.

Formal review stages:

1. Planning: define objectives, scope, participants, schedule, and entry criteria.
2. Overview: author explains purpose, design, interfaces, and high-risk areas.
3. Individual preparation: reviewers examine code independently.
4. Review meeting: discuss evidence-based findings, not personal opinions; record issues without turning the meeting into a lengthy solution-design session.
5. Rework: author corrects accepted defects.
6. Follow-up: moderator checks corrections and confirms exit criteria.

Review checklist:

- Naming conventions and consistent formatting.

- Duplicate logic and unnecessary complexity.
- Function length, cohesion, and clear responsibilities.
- Correct conditions, loops, and boundary handling.
- Input validation and safe data handling.
- Exception handling and useful error messages.
- Resource cleanup and concurrency risks.
- Security, access checks, and secret handling.
- Comments that explain intent and match the code.
- Testability, maintainability, and requirement traceability.

Defect classification:

- Critical: security exposure, data corruption, or major business failure.
- Major: important function incorrect or blocked.
- Minor: limited functional or maintainability issue.
- Observation: suggestion or style improvement.

Severity should describe impact; priority should describe urgency.

Documentation and follow-up: Keep a review log with issue ID, location, description, category, severity, owner, and status. Record review date, participants, metrics, and decision. Re-review changed areas, verify fixes, and close only when required corrections are complete. The review is complete when exit criteria are met and remaining issues are accepted and documented.

## 4. Differentiate peer review and formal review; design a peer review for an e-commerce critical module.

Peer review is a broad term for colleagues examining work products to find issues and improve quality. It can be informal, such as a pull-request review. A formal review follows defined roles, preparation, meetings, records, and exit criteria.

Peer review:

- Usually lighter and quicker.
- May use comments in a code-review tool.
- Procedures and documentation can vary.
- Suitable for regular changes and shared code ownership.

Formal review:

- Has a defined process, roles, checklist, and records.
- Includes planned preparation and follow-up.
- Better suited to high-risk or regulated components.
- Provides stronger traceability and audit evidence.

Example: Five developers are building an e-commerce application. For the payment/order-total module, one developer authors the change; two developers review business logic and code; one checks security and payment handling; and one reviewer or team lead coordinates the process. Roles can rotate.

Procedure:

1. Author opens a pull request and links requirements and tests.
2. Reviewers independently check price calculations, discount boundaries, currency precision, duplicate payment requests, authorization, logging, and error handling.
3. Reviewers record each finding with file/line, explanation, and severity.
4. The team discusses unclear findings respectfully and agrees whether they are defects or suggestions.

5. Author fixes accepted defects and adds or updates automated tests.
6. Reviewers inspect the changes and run relevant tests.
7. Merge only after required approvals and checks pass.
8. Record unresolved risks and monitor the module after integration.

This procedure catches defects before they reach integration and creates a review history.

## 5. Prepare a coding-review checklist with at least 20 criteria and explain their importance.

Use the following checklist during code review:

1. Meaningful variable names — make intent easy to understand.
2. Consistent naming conventions — improve readability across the codebase.
3. Consistent formatting and indentation — make logic easier to inspect.
4. Small, focused functions — reduce complexity and simplify testing.
5. Single responsibility — keep each function or class focused on one purpose.
6. Avoid duplicated logic — reduce inconsistent fixes and maintenance effort.
7. Avoid unexplained constants — named constants clarify business rules.
8. Clear conditional logic — reduce mistakes in nested conditions and branches.
9. Correct loop boundaries — prevent skipped items and off-by-one defects.
10. Validate all external inputs — stop malformed data entering core logic.
11. Check ranges and boundary values — prevent invalid or extreme values.
12. Handle null/empty values — avoid unexpected crashes.
13. Use specific exception handling — prevent errors from being silently hidden.
14. Provide safe error messages — help users without exposing internal details.
15. Release resources reliably — close files, connections, and streams to prevent leaks.
16. Use parameterized database queries — reduce SQL injection risk.
17. Apply authorization checks — ensure users access only permitted records/actions.
18. Protect secrets — never hard-code passwords, keys, or tokens.
19. Avoid sensitive data in logs — protect personal and financial information.
20. Use secure password/cryptographic practices — avoid weak storage or outdated algorithms.
21. Check concurrency and shared state — prevent race conditions and inconsistent data.
22. Optimize only where justified — avoid unnecessary performance costs without harming clarity.
23. Use appropriate data structures and indexes — support efficient processing.
24. Write useful comments — explain reasoning, assumptions, and non-obvious decisions.
25. Keep comments accurate — outdated comments can mislead future maintainers.
26. Add unit tests for normal and edge cases — verify intended behaviour.
27. Test failure paths — ensure the module handles errors safely.
28. Maintain requirement traceability — connect code and tests to business rules.
29. Follow project standards and lint rules — create predictable, consistent code.
30. Check dependency and configuration changes — avoid insecure or incompatible versions.

Reviewers should record real defects separately from optional style suggestions and prioritize issues according to impact and risk.

## 6. Design Equivalence Partitioning for a Student Result Management System.

Assumptions: Each of five subjects accepts integer marks from 0 to 100 inclusive. For illustration, assume passing requires at least 40 marks in every subject. Grade thresholds are not provided, so grade expectations must be confirmed from the university's rules before testing.

Equivalence Partitioning divides inputs into classes expected to behave similarly.

For each subject:

- Valid partition: integer 0–100.
- Invalid low partition: value below 0.
- Invalid high partition: value above 100.
- Invalid type partition: text, decimal if only integers are allowed, or a non-numeric value.
- Missing partition: no mark supplied.
- Malformed partition: symbols or improperly formatted values.

Representative tests and expected results:

1. Marks [60, 70, 80, 90, 100]: accept all marks; total 400; percentage 80%; all subjects pass. Grade follows the official grading rule.
2. Marks [40, 40, 40, 40, 40]: accept boundary values; total 200; percentage 40%; pass if 40 is the minimum pass mark.
3. Marks [0, 50, 50, 50, 50]: accept zero; total 200; percentage 40%; fail under the assumed rule because one subject is below 40.
4. One mark −1, others valid: reject the invalid mark and do not calculate a final result.
5. One mark 101: reject.
6. One mark "abc": reject with a clear validation message.
7. One mark missing: request the missing mark; do not produce a final result.
8. One mark 45.5: reject if only integer marks are allowed; otherwise test according to the official rule.
9. All marks 100: total 500; percentage 100%; all subjects pass.
10. All marks 0: total 0; percentage 0%; fail.

Why sufficient: Representative tests cover valid, low-invalid, high-invalid, missing, and wrong-type partitions. Since each subject has the same rule, test each subject at least once for field-specific mistakes, then test combined records for total, percentage, grade, and pass/fail logic. Boundary values should also be tested explicitly because partitioning alone may miss boundary errors. Confirm the actual grade bands and pass rules with the requirements.

## 7. Design Equivalence Partitioning and Boundary Value Analysis for shopping discounts.

Assumption: The purchase amount is a non-negative monetary value in rupees, and discount bands are inclusive at both stated ends. The rules are: below ₹1,000 = 0%; ₹1,000–₹4,999 = 5%; ₹5,000–₹9,999 = 10%; ₹10,000–₹49,999 = 15%; ₹50,000 or above = 20%. Confirm whether the system uses whole rupees or paise and how rounding is handled.

Equivalence classes:

- Invalid: negative amount, missing amount, non-numeric input.
- Class 1: ₹0–₹999.99 → 0%.
- Class 2: ₹1,000–₹4,999.99 → 5%.
- Class 3: ₹5,000–₹9,999.99 → 10%.
- Class 4: ₹10,000–₹49,999.99 → 15%.
- Class 5: ₹50,000 and above → 20%.

Boundary tests (amount → expected discount rate):

₹999.99 → 0%; ₹1,000.00 → 5%; ₹1,000.01 → 5%.

₹4,999.99 → 5%; ₹5,000.00 → 10%; ₹5,000.01 → 10%.

₹9,999.99 → 10%; ₹10,000.00 → 15%; ₹10,000.01 → 15%.

₹49,999.99 → 15%; ₹50,000.00 → 20%; ₹50,000.01 → 20%.

Also test ₹0 → 0%, −₹1 → rejected, blank → validation error, and text input → validation error.

If the system accepts only whole rupees, test 999, 1,000, 1,001; 4,998, 4,999, 5,000; 4,999/5,000 and corresponding neighbours at 10,000 and 50,000. Use the exact domain defined by requirements.

Potential defects: using < instead of ≤, gaps or overlaps between bands, applying the next band too early, incorrect rate, rounding errors, negative amounts receiving a discount, and inconsistent handling of paise. Verify both discount rate and final payable amount. For example, ₹5,000 at 10% should produce a ₹500 discount and ₹4,500 before any taxes or other charges.

## 8. Explain Test to Pass and Test to Fail using an ATM withdrawal system.

Test to Pass checks whether the ATM works correctly with valid inputs and normal conditions. Test to Fail deliberately uses invalid inputs or failure conditions to confirm that the system rejects them safely.

Test to Pass scenarios:

1. Valid card, correct PIN, active account, sufficient balance, and available cash: dispense requested cash and update balance once.
2. Withdraw an amount within the allowed limit: transaction succeeds and receipt is accurate.
3. Withdraw the minimum permitted amount: succeeds if balance and denomination rules allow it.
4. Enter a valid PIN and cancel before confirmation: no cash is dispensed and no debit occurs.
5. Complete withdrawal when network and bank service are available: show success and record the transaction.

Test to Fail scenarios:

1. Incorrect PIN: deny access, record the failed attempt, and follow the lockout policy.
2. Blocked/expired card: reject the transaction without revealing sensitive account information.
3. Insufficient balance: reject withdrawal and do not debit the account.
4. Amount below minimum, above daily limit, or not supported by available denominations: reject with a suitable message.
5. Invalid input such as letters, negative amount, zero, or malformed value: validate and reject.
6. Network failure before authorization: do not dispense cash or create an incorrect debit.
7. Network failure after authorization or during response: reconcile transaction status; avoid duplicate debit or cash dispensing.
8. Transaction timeout: show a clear status and ensure recovery/reconciliation prevents double withdrawal.
9. ATM has insufficient cash: cancel safely and do not debit the account.
10. Repeated confirmation or retry: use transaction identifiers/idempotency so the same request is not processed twice.

Boundary tests should cover the minimum withdrawal, maximum per-transaction amount, daily limit, available balance exactly equal to the requested amount, and one unit above or below each limit. Test outcomes must follow the bank's documented rules. Both approaches are required: successful paths prove intended behaviour, while failure paths check safety, correctness, and recovery.

## 9. Design a complete Data Testing strategy for a hospital appointment system.

Data testing checks whether data is valid, complete, consistent, secure, and correctly stored or processed.

1. Valid data: realistic patient age, an existing doctor, a future available appointment, and a successful payment. Expected: booking is created once and linked to the correct patient, doctor, date, and payment.
2. Invalid data: age below zero, nonexistent doctor ID, appointment in the past, or unsupported payment status. Expected: reject with a clear message.
3. Boundary data: age at 0, 1, the highest medically/business-supported age, and values just outside allowed limits; appointment at opening/closing dates and times; payment at zero or exact fee. Define age limits from policy rather than guessing.
4. Missing data: blank age, doctor, date, patient identifier, or payment reference. Required fields must be rejected; optional fields may remain blank.
5. Incorrect data types: letters in numeric age, invalid date strings, malformed identifiers, or text in a payment amount. Validate types and formats.
6. Duplicate data: repeated booking submissions, duplicate payment callbacks, or the same appointment request sent twice. Ensure idempotency and prevent unintended duplicate bookings or charges.
7. Inconsistent data: booking marked paid while payment is failed; appointment assigned to an unavailable doctor; conflicting time zones; or patient ID not matching the record. Enforce consistency and transactional updates.
8. Extreme values: unusually long names, very large numeric input, leap-day dates, maximum supported string lengths, and large concurrent booking volumes. Reject or handle safely without crashing.
9. Malicious/unexpected input: SQL injection strings, script tags, path-like input, Unicode edge cases, and oversized payloads. Use parameterized queries, output encoding, validation, and size limits.
10. Privacy and access checks: ensure users cannot view or change another patient's records; avoid exposing health or payment details in logs.
11. Database integrity: verify required fields, foreign keys, uniqueness constraints, timestamps, and rollback after failed transactions.
12. Recovery: simulate payment timeout, network loss, and service restart; confirm the booking and payment state can be reconciled.

Record test data, expected outcomes, observed results, and defects. Use synthetic or appropriately de-identified patient data and restrict access to sensitive information.

## 10. Develop a State Transition Testing model for a bank account application.

State Transition Testing checks how a system behaves when events cause it to move between defined states.

States:

- New Account: created but not yet activated.
- Active: permitted account operations.
- Suspended: restricted because of suspicious activity or a policy condition.
- Closed: permanently closed and cannot return to Active.

Model:

New Account --activation approved--> Active

Active --suspicious activity--> Suspended

Suspended --review cleared--> Active

Active --closure conditions satisfied--> Closed

Suspended --closure conditions satisfied, if policy allows--> Closed

Valid transition test cases:

1. Create account → New Account; verify it cannot perform restricted transactions before activation.
2. Complete required verification → Active; verify allowed operations.
3. Active account triggers suspicious activity → Suspended; verify transactions are blocked or restricted according to policy and an alert/audit record is created.

4. Suspended account passes review → Active; verify permissions are restored.
5. Active account meets closure conditions and closure is confirmed → Closed; verify account cannot transact and closure is recorded.
6. Suspended account meets permitted closure conditions → Closed, if the business rules allow this transition.

Invalid transition tests:

- New Account → Closed without satisfying closure rules.
- New Account → Suspended if no rule permits it.
- Suspended → Active without a successful review.
- Closed → Active; should be rejected because closure is permanent.
- Any transaction that is forbidden in Suspended or Closed state.

Test repeated events, simultaneous activation/suspension requests, stale status data, and service failures. Verify state, permissions, notifications, audit history, and related records after each event. Exact transitions must be confirmed by the bank's requirements.

## 11. Design state-based testing for a Food Delivery Application.

Primary states: Order Placed → Payment Confirmed → Restaurant Accepted → Food Preparing → Out for Delivery → Delivered. Alternative states include Payment Failed, Cancelled, and Refund Initiated.

Valid transitions:

- Order Placed → Payment Confirmed after successful payment.
- Order Placed → Payment Failed after a failed payment attempt.
- Payment Confirmed → Restaurant Accepted when the restaurant accepts.
- Restaurant Accepted → Food Preparing.
- Food Preparing → Out for Delivery.
- Out for Delivery → Delivered after delivery confirmation.
- Order Placed → Cancelled if cancellation is permitted.
- Payment Confirmed → Cancelled if policy permits and cancellation is accepted.
- Cancelled or eligible failed-payment state → Refund Initiated when money was captured and a refund is due.
- Refund Initiated → Refund Completed/Failed if these states are defined by the system.

Invalid transitions:

- Payment Failed → Food Preparing without a valid payment or approved cash-on-delivery rule.
- Delivered → Food Preparing.
- Cancelled → Out for Delivery.
- Refund Initiated → Delivered as a normal state transition.
- Skipping required states without an explicitly supported exception.

Unexpected and repeated events:

- Duplicate payment confirmation or restaurant acceptance.
- Customer cancels while food is being prepared.
- Restaurant rejects after payment.
- Driver becomes unavailable or delivery is delayed.
- Payment callback arrives after timeout.
- Duplicate delivery confirmation or refund request.

- Two status updates arrive out of order.

Recovery scenarios:

- Payment succeeds at the gateway but the app times out; reconcile status without charging twice.
- Restaurant rejects after payment; cancel and initiate refund according to policy.
- Network loss during status update; retry safely and retain a consistent state.
- Refund service fails; keep the refund pending and retry without issuing duplicates.

For every test, verify the current state, allowed actions, customer notifications, payment/refund records, timestamps, and audit log. State rules and cancellation cutoffs should come from business requirements.

## 12. Apply Decision Table Testing to an insurance claim processing system.

Decision Table Testing checks combinations of conditions and the action that should result. Assume approval requires all five conditions: policy valid, premium paid, accident covered, required documents available, and claim amount within the policy limit.

Conditions:

C1 Policy valid? C2 Premium paid? C3 Accident covered? C4 Documents complete? C5 Amount within limit?

Decision table (Y = yes, N = no):

Rule 1: Y, Y, Y, Y, Y → Approve claim.

Rule 2: N, –, –, –, – → Reject or refer according to policy.

Rule 3: Y, N, –, –, – → Reject or hold for payment review.

Rule 4: Y, Y, N, –, – → Reject as not covered.

Rule 5: Y, Y, Y, N, – → Request missing documents / keep pending.

Rule 6: Y, Y, Y, Y, N → Reject or send for special review, according to policy.

A dash means the condition does not affect that rule's outcome. If the business rules permit exceptions, the table must be expanded to include them.

Derived test cases:

1. All conditions Yes → approve.
2. Policy invalid → reject/refer.
3. Policy valid but premium unpaid → reject/hold.
4. Valid policy and paid premium but accident excluded → reject.
5. All conditions met except documents missing → pending and request documents.
6. All conditions met except amount exceeds limit → reject or special review.

Decision tables expose missing or contradictory business rules. Test each rule with representative data and verify both the decision and any notifications, audit records, and status updates.

## 13. Analyze a login system using Decision Table Testing.

Login conditions:

C1 Username registered? C2 Password correct? C3 Account active? C4 Account not locked? C5 CAPTCHA valid?

A login succeeds only when all five conditions are Yes.

Decision table:

Rule 1: Y, Y, Y, Y, Y → Login allowed.

Rule 2: N, –, –, –, – → Reject login.

Rule 3: Y, N, –, –, – → Reject login and apply failed-attempt policy.

Rule 4: Y, Y, N, –, – → Reject login because account is inactive.

Rule 5: Y, Y, Y, N, – → Reject login because account is locked.

Rule 6: Y, Y, Y, Y, N → Reject login because CAPTCHA failed.

A dash means later conditions need not be evaluated for the stated decision, although implementation and security requirements may require uniform external error messages.

Minimum effective tests:

1. All valid conditions → successful login.
2. Unregistered username → rejected.
3. Registered username with wrong password → rejected.
4. Correct credentials for inactive account → rejected.
5. Correct credentials for locked account → rejected.
6. Correct credentials but invalid/expired CAPTCHA → rejected.
7. Missing username or password → validation error.
8. Repeated wrong passwords → verify lockout/rate-limit policy.
9. CAPTCHA replay or expired token → rejected.
10. Concurrent or repeated submissions → no unintended bypass.

The first six cover each decision rule. Additional negative and security tests check robustness, lockout, and CAPTCHA implementation. Verify that failures do not reveal whether a username exists unless explicitly intended by the security design.

## 14. Develop a black-box test suite for password requirements.

Requirements: password length 8–20 characters inclusive, with at least one uppercase letter, one lowercase letter, one digit, and one special character. Assume spaces are allowed only if the product specification says so; define the accepted special-character set explicitly.

Equivalence Partitioning:

- Valid: length 8–20 and all four character categories present.
- Invalid short: length 0–7.
- Invalid long: length above 20.
- Invalid composition: missing uppercase, missing lowercase, missing digit, or missing special character.
- Missing password: empty or null.
- Invalid format: unsupported characters or encoding, if restricted by requirements.

Boundary Value Analysis:

- 7 characters → reject.
- 8 characters with all required categories → accept.
- 9 characters with all required categories → accept.
- 19 characters with all required categories → accept.
- 20 characters with all required categories → accept.
- 21 characters → reject.

Example valid passwords (illustrative only; do not reuse them as real credentials): `Abcdef1!`, `StrongPass8#XY`, and a 20-character string that includes all categories. Confirm each sample's exact length before use.

Negative tests:

1. `abcdef1!` — no uppercase; reject.
2. `ABCDEF1!` — no lowercase; reject.
3. `Abcdefg!` — no digit; reject.

4. `Abcdefg1` — no special character; reject.
5. 7-character password with all categories — reject for length.
6. 21-character password with all categories — reject for length.
7. Empty input — reject.
8. Very long input — reject safely without crash or truncation bypass.
9. Unicode and allowed special characters — behave according to the documented character policy.
10. Leading/trailing spaces — apply the specified trim/reject policy consistently.

Defects targeted: off-by-one length checks, missing category validation, accepting empty input, rejecting valid special characters, silent truncation, and inconsistent Unicode handling. Also test that password values are not logged and that storage uses an approved password-hashing method. Do not assume any password sample is secure for actual use.

## 15. Explain static black-box testing and review an online-shopping requirement document.

Static black-box testing reviews requirements and specifications without running code and without relying on internal implementation details. The reviewer checks whether the documented external behaviour is clear, complete, consistent, feasible, and testable.

Sample requirement statements and problems:

1. Ambiguous: “The website should load quickly.” Problem: “quickly” is not measurable. Improve to a defined response-time target under stated load and environment.
2. Incomplete: “Users can pay online.” Problem: no supported payment methods, failure handling, duplicate-payment rule, or refund behaviour. Add these details.
3. Contradictory: “Orders can be cancelled at any time” and “Orders cannot be cancelled after dispatch.” Problem: conflicting rules. Clarify the cancellation cutoff and exceptions.
4. Unverifiable: “The checkout process must be user-friendly.” Problem: no measurable acceptance criteria. Define usability tasks or success criteria.
5. Missing: No rule describes what happens if payment succeeds at the gateway but the website times out. Add reconciliation, retry, and duplicate-prevention requirements.
6. Ambiguous discount requirement: “Large orders receive a discount.” Define thresholds, rates, eligible products, and rounding.
7. Incomplete inventory requirement: “Items are added to the cart.” Clarify stock reservation, out-of-stock behaviour, and what happens if stock changes before payment.

Pre-implementation review checklist:

- Is each requirement uniquely identified?
- Is the language clear and unambiguous?
- Is every term defined consistently?
- Are normal, alternative, and failure flows described?
- Are inputs, outputs, limits, and data formats specified?
- Are calculations, discounts, taxes, and rounding rules explicit?
- Are authentication, authorization, privacy, and security needs defined?
- Are payment success, failure, timeout, retry, and refund rules covered?
- Are inventory, cart, order, and payment states consistent?
- Are cancellation, return, and delivery rules complete?
- Are performance, availability, compatibility, and accessibility requirements measurable?
- Are error messages and recovery behaviour specified?
- Can each requirement be tested objectively?

- Are acceptance criteria and priorities included?
- Are dependencies, assumptions, and exclusions documented?
- Are requirements traceable to design and test cases?
- Are contradictory rules resolved and stakeholders in agreement?

Importance: Finding specification defects before implementation prevents incorrect designs and code, reduces rework, and makes later test cases more accurate. Static black-box testing complements code reviews and dynamic testing; it does not replace them.